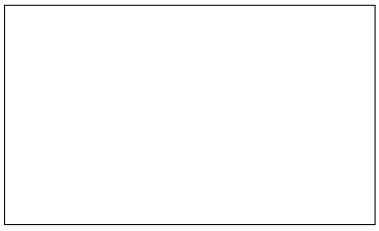
Graphical Abstract

{TODO: Change title to reflect intro changes}SolidSessionBench: Realistic Query Sequences for User-Oriented Decentralized Environments

Ruben Eschauzier, Ruben Taelman



Highlights

{TODO: Change title to reflect intro changes}SolidSessionBench: Realistic Query Sequences for User-Oriented Decentralized Environments

Ruben Eschauzier, Ruben Taelman

- Research highlights item 1
- Research highlights item 2
- Research highlights item 3

{TODO: Change title to reflect intro changes}SolidSessionBench: Realistic Query Sequences for User-Oriented Decentralized Environments

Ruben Eschauzier[a,*], Ruben Taelman[a]

[a]Department of Electronics and Information Systems, Ghent University – imec, Universiteitstraat 4, Ghent, 9000, Flanders, Belgium

ARTICLE INFO

Keywords:
quadrupole exciton
polariton
WGM
BEC

ABSTRACT

To build user-facing applications on decentralized data, client-side decentralized query engines offer a natural integration point. However, optimizing queries in these environments is challenging due to a lack of prior knowledge regarding data distribution and availability. By repeatedly interacting with the same engine from the same user, applications allow the engine to overcome this knowledge gap by exploiting patterns in previously issued queries. However, evaluating these techniques accurately is difficult because public query logs are scarce, and existing benchmarks rely on static, template-based structures that fail to capture realistic behavioral variability. To fill this gap, we introduce an evidence-based simulation of client query usage patterns as a reusable benchmark enabling realistic and reproducible evaluation of client-side optimization techniques. We demonstrate its utility by evaluating client-side caching strategies for link traversal query processing. By comparing our sequence-based approach against traditional, non-sequence-based workloads, we highlight how realistic sequence simulation heavily influences performance evaluations. Ultimately, our benchmark reveals important performance dynamics, such as the impact of session switching and query refinement, that traditional workloads do not. We conclude that modeling realistic user interactions provides an essential foundation for accurately evaluating client-side optimization techniques in decentralized environments.
{TODO: Abstract should reflect changes in introduction framing} `\begin{keyword} ... \end{keyword}` which contain the abstract and keywords respectively.
Each keyword shall be separated by a `\sep` command.

Journal extension

- ☐ Rework introduction to fit special issue in some way
- ☐ Add explanation of different approaches we added
- ☐ Extend same results we have with disk, offline traversal, and quad store

1. Introduction

Emerging web applications increasingly rely on decentralized architectures to give users control over their personal data. Structured decentralized ecosystems, such as Solid [49] and Mastodon [46], encourage users to keep their data in small, independent stores following shared data models. This paradigm shift is increasingly vital for automated systems, such as LLM-based AI agents, which query these decentralized structures to extract facts for task performance or question answering [61, 34, 10]. While this approach strengthens user autonomy, the absence of a central data authority naturally creates a highly multi-perspective environment. Data spreads across thousands of heterogeneous sources that frequently contain overlapping, diverse, or conflicting claims. Query engines must therefore dynamically fetch and integrate this information at runtime, not only to discover conflicting statements and multiple perspectives, but to reconcile these claims to provide a coherent answer to the user or agent.

Traditional federated querying approaches [52, 50, 32] support decentralization of sources, but typically assume a small (10-100) set of well-described and consistently available endpoints [17]. These assumptions fail once data is spread across thousands of small units that enforce local access-control policies. In addition, enforcing fine-grained access control policies [21] on large sources with expressive interfaces introduces notable overhead [43], making standard approaches unsuitable for highly decentralized environments.

Newer decentralized environments, such as Solid, often expose data as lightweight HTTP resources instead of highly expressive query interfaces. Identifying and resolving inconsistencies across these networks requires a querying approach capable of navigating resources that contain conflicting information. Many of these resources are not known in advance and can only be discovered during query execution. While their simplicity lowers the cost of enforcing fine-grained controls, this setting requires a federation model that can operate over a very large number of small sources, many of which may be encountered at runtime, without relying on prior aggregation. Traditional federated approaches are insufficient in this setting due to the assumption of a small number of large endpoints. Executing SPARQL queries using Link Traversal-based Query Processing (LTQP) [30]

ruben.eschauzier@gmail.com (R. Eschauzier); ruben.taelman@ugent.be (R. Taelman)
www.rubensworks.net (R. Taelman)
ORCID(s): 0000-0001-0000-0000 (R. Eschauzier); 0000-0001-5118-256X (R. Taelman)

fits these needs. It discovers documents incrementally by following URIs encountered during execution, starting from an initial set of seed documents provided by either the user or the query. Its link-driven, asynchronous traversal supports fine-grained permissions and allows systems to discover and reconcile conflicting information without knowing the location of all decentralized data sources beforehand.

However, the fact that LTQP discovers documents incrementally creates significant challenges for efficient execution. Using the traditional optimize-then-execute paradigm is difficult, as the query itself determines which data will be accessed. As a result, the optimizer has no prior knowledge of the relevant data until the query's traversal has already begun. Some approaches optimize query execution without prior knowledge by using adaptive query processing [26] and zero-knowledge query planning [28]. These yield modest but inconsistent performance gains, depending on the decentralized environment and the heuristics used [56]. However, other aspects of zero-knowledge LTQP optimization, such as link prioritization [31], are unlikely to improve query result arrival times in emerging decentralized environments [20].

Consequently, other works explore using precomputed server-side information as prior knowledge or relevance indexes to efficiently execute LTQP queries [57, 58, 25]. While these approaches offer greater performance benefits, they require the server to maintain these indexes. This raises the cost of serving resources and complicates privacy and access control.

To avoid server-side overhead, engines can derive prior knowledge directly on the client. In decentralized settings, LTQP is inherently client-side, with a single engine instance serving one or more applications. As these applications issue sequences of correlated queries, the engine identifies frequently accessed data and its characteristics. This allows engines to implement *personalized* query optimization algorithms that leverage local access patterns to build necessary prior knowledge.

{TODO: Adjust figure to reflect fact that AI is no longer focus**}**

Benchmarking these approaches requires realistic query sequences. Traditional benchmarks (e.g., WatDiv [1], BSBM [11]) execute isolated or repeated query templates, which fails to capture client-side dynamics accurately. Furthermore, while centralized SPARQL optimization relies on real query logs [9, 45, 12, 53], such logs do not yet exist for decentralized LTQP environments. Consequently, we need a standardized benchmark that realistically simulates user query sequences to prevent fragmented evaluation practices.

In this paper, we address this gap by introducing SolidSessionBench, a benchmark extending SolidBench [56] to generate realistic query sequences. These sequences capture usage patterns identified in social network literature, a domain naturally rich in multi-perspective data, alongside patterns observed in real-world SPARQL logs.

To evaluate the realism of our generated query sequences without access to real-world query logs, we investigate **{TODO**: insert**}** and evaluate **{TODO**: insert**}** baseline client-side caching strategies in the LTQP engine Comunica [54, 56]. We use these simple algorithms to establish a performance baseline and demonstrate the necessity of sequence-based evaluation. Comparing this baseline against existing literature shows that current evaluation techniques are insufficient for client-side caching in LTQP.

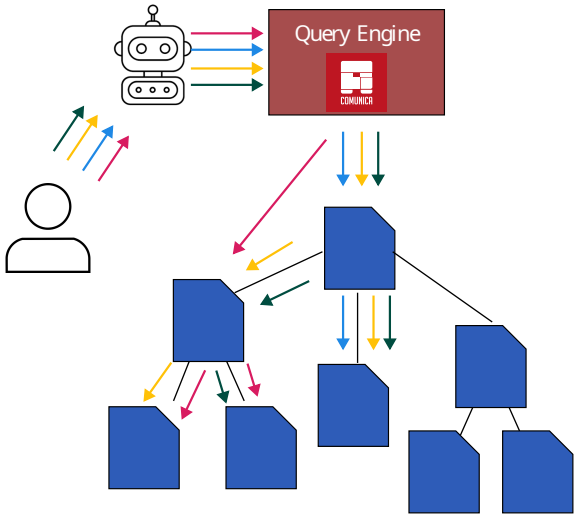


Figure 1: When users interact with agents, their queries intentions and thus relevant data to the query can overlap, leading to opportunities for personalized optimization.

By enabling researchers to evaluate personalized query optimization against actual user behaviors, SolidSessionBench lays the foundation for performant client-side querying. Ultimately, efficient LTQP execution over decentralized sources facilitates the rapid identification of conflicting information and the application of consistency fixes across multi-dimensional knowledge graphs.

The remainder of this paper is structured as follows. Section 2 reviews prior benchmarking approaches and real-world query usage patterns. Section 3 outlines our simulation methodology and defines the resulting hyperparameters. Section 4 introduces the technical mechanics of client-side caching strategies in LTQP. Section 5 evaluates the benchmark using proxy measures of realism. Finally, Section 6 concludes the paper and presents a plan for benchmark sustainability.

{TODO: Maybe insert section summarizing contributions.**}**

2. Literature Review

To ground our benchmark design in existing research and ensure a realistic sequence generator, this section reviews prior benchmarking approaches for client-side query optimization and real-world query usage patterns.

2.1. Existing Benchmarking Approaches

For the evaluation of client-side benchmarking techniques that rely on previous queries, we primarily investigate

caching literature, as this is a well-studied problem relevant to our work.

In the literature, we find three distinct benchmarking approaches: simulated micro benchmarks, simulated end-to-end query benchmarks, and real-world query logs.

Simulated micro benchmarks attempt to isolate performance gains of the benchmarked approaches by fixing certain aspects of query execution. This can range from fixing the cached data size [27] to controlling the percentage of query-relevant data present in the cache [41]. Fixing aspects of query execution makes it easier to isolate and understand the effect of specific caching mechanisms. However, it also gives an incomplete picture, because we don't know how those fixed conditions relate to real workloads or how the methods perform when those assumptions don't hold.

Simulated end-to-end query benchmarks use generated query sequences to evaluate performance. They aim to measure cache hit rate and overall execution behavior under the assumption that the simulated sequences reflect realistic usage. In practice, the simulation is usually built by extending an existing benchmark so that it generates sequences instead of isolated queries. The sequence order is often determined by simple rules or by random selection.

A common approach is to use queries from existing benchmarks such as the Star Schema Benchmark [42] or the Berlin SPARQL Benchmark [11]. These queries are then randomly divided into sequences [27, 41, 23], with each sequence executed using a shared cache. This approach is straightforward but not realistic. Prior work shows that real workloads contain user behavior patterns and temporal locality, and these patterns can be exploited by caching methods. Random sequences fail to capture these properties.

More realistic is to adjust the query generators of existing benchmarks to include patterns in the query sequences. Some works [62, 37] use a Pareto distribution to decide what entity is used to instantiate a query template. This follows from the observation that many human activities follow a Pareto distribution [6], with many queries relating to a small portion of the entities.

A third approach uses simulated query sequences that reflect the access patterns of specific applications. One example is a workload that models data scientists exploring different aspects of the same entity—such as an author in a scholarly network—through consecutive, thematically related queries [15]. Such simulations often include multiple query sessions, with new sessions beginning according to a probability parameter p. Another example is a simulated application scenario focused on retrieving information about vacation destinations, which is used to assess caching effectiveness [37].

Real-world query logs Whenever available, query logs from actual applications form a highly relevant benchmarking approach. However, obtaining these query logs can be challenging. For SPARQL query optimization, query logs for a variety of endpoints are available, such as DBpedia [48, 53, 9] and Wikidata [53]. These logs are extensively used in benchmarking SPARQL-based approaches [63, 44, 35].

The evaluation of personalized LTQP optimization would ideally use real-world query logs, but such logs do not exist. To avoid the fragmented evaluation practices of current simulated approaches, the next best option is a unified simulated benchmark. A shared benchmark would provide a common basis for comparison, improve reproducibility, and prevent each paper from recreating its own evaluation setup.

2.2. Real-world Query Usage Patterns

To inform the benchmark design on what patterns should be simulated, we will outline three relevant client query usage patterns observed in the literature. The relevance is determined for our social network application use case.

User Interest-Based Query Behavior Work on social networks shows users form communities based on shared interests [39, 16, 7, 33]. These networks exhibit power-law degree distributions, small-world structures, and scale-free properties. This topology creates a dense core of high-degree nodes that connect many smaller, strongly clustered groups [39]. Simulation studies confirm that this interest-driven group formation is stable and reliably modeled [2, 24].

Furthermore, empirical research formally establishes that shared interests directly drive a higher degree of explicit and implicit interactions between users [40]. These findings justify our simulation of user-relatedness and interest-driven behavior when generating query sequences.

Logical Query Sessions Analysis of web browsing behavior shows that activity is best grouped into *logical sessions*: sequences of user interactions focused on specific goals, rather than purely time-bound intervals [38, 47, 51]. These task-oriented sessions frequently involve non-linear navigation, including backtracking. New sessions begin in approximately 15% of clicks, following a log-normal distribution with average size and depth means of 6.1 and 3, respectively.

These patterns also appear in application navigation. In a decentralized social media application, for example, a user viewing a feed might click a creator's username, view comments, or like a post. Because the parameters of each new query derive directly from the preceding query's results, these queries chain together into structures mirroring logical web sessions. Additionally, direct searches for specific posts or topics reflect standard session-switching behavior.

Consequently, interactive applications generate sequences of interdependent requests [60]. This dynamic informs the design of Facebook's TAO data store, which is explicitly optimized for workloads driven by the user's step-by-step traversal of the social graph within the interface [14].

Query Refinement Analysis of SPARQL queries issued to the DBpedia endpoint reveals that users frequently submit related queries in streaks [13, 64]. These streaks consist of query sequences that share an underlying intention but undergo iterative refinement to achieve the user's objective. Previous studies initially introduced the concept of SPARQL query sessions specifically to analyze robotic queries [64, 13].

In this benchmark, we focus on organic query sequences [64], which users generate through interactions with software or browsers [36]. Because organic interactions are driven by application design rather than underlying data organization, we assume these user behaviors will also emerge in applications navigating structured decentralized environments Furthermore, we deprioritize robotic queries because their sequences can already be simulated by traditional benchmark approaches [56, 11, 1].

For both organic and robotic queries, analysis of total usage and reformulations (removals and additions) of operators over all query logs shows that filter, union, and optional operators account for the vast majority of reformulated queries.

Recent analyses of Wikidata query logs further demonstrate that query development is an iterative process of design, debugging, and maintenance [5]. Based on this empirical data, exploratory querying encompasses six core behaviors [5]: **result refinement** to reduce records via filters or selective triple patterns, **result generalization** to retrieve more entities by broadening parameters, such as adding superclasses or increasing limits, **result extension** to fetch additional attributes by appending new triple patterns, **bug fixing** to correct structural errors like invalid URIs, **query refinement** to adjust complex language features, and **shifting query intent** to pivot topics by altering core predicates.

These findings confirm that real-world querying relies on dynamic, step-by-step modifications rather than the static execution of independent templates. Consequently, existing evaluation frameworks fail to capture how users actively interact with data systems. Evaluating system performance for these exploratory query sequences remains a critical research challenge, explicitly requiring new benchmarks to validate progress [5]. Therefore, modeling these empirical sequence patterns is essential to accurately assess client-side query optimization techniques.

3. Query Sequence Benchmark

In this section, we detail our simulation methodology and the resulting hyperparameters. We base our sequence generator on SolidBench [56], which simulates realistic decentralized social media applications within the Solid ecosystem. SolidBench utilizes the Social Network Benchmark (SNB) dataset [19, 3] and applies the Linked Data Benchmark Council (LDBC) choke point methodology [4]. Its workload combines standard SNB *interactive* queries with custom *discover* templates, which explicitly target Link Traversal Query Processing (LTQP) choke points in decentralized environments.

To support sequence-based evaluation, we extend three core components of the SolidBench ecosystem:

- **Query Generator:** We modify the generator to produce correlated query sequences that reflect empirical real-world usage patterns.
- **Dataset Generator:** We enhance the data generation process to model interest-based similarities between simulated users.
- **Benchmark Runner:** We integrate sequence execution logic into the existing runner to facilitate straightforward, reproducible sequence-based experiments and allow easy generation of default configuration files.

3.1. Simulation Methodology

Our methodology models three interdependent behaviors: task-oriented logical sessions, user interests for data locality, and refinement patterns for iterative query development.

3.1.1. Logical Query Sessions

To model logical sessions, we categorize queries into four primary topics: person-specific messages, forum information, person attributes, and message attributes. Because queries in decentralized environments often depend on preceding results, we map the output types of each template to valid subsequent templates. This creates a directed graph of query progression that governs the flow of a logical session.

Within a session, a template's *instantiation values* are typically derived from the previous query's results. To realistically simulate this "click-through" behavior, we execute centralized equivalents of the LDBC SNB queries using QLever [8]. We then randomly sample the returned result set to instantiate the subsequent query template. If a query yields no results, we fall back to our default, interest-based instantiation methodology.

As an example, consider the following data distributed across a decentralized environment:

Listing 1: A minimal decentralized dataset distributed across documents.

```
@prefix : <https://example.org/voc#> .

# File: https://pod.example/alice/profile.ttl
<https://pod.example/alice/profile.ttl#me> a :Person ;
    :knows <https://pod.example/bob/profile.ttl#me> ;
    :hasInbox <https://pod.example/alice/inbox> .

# File: https://pod.example/bob/profile.ttl
<https://pod.example/bob/profile.ttl#me> a :Person ;
    :posts <https://pod.example/bob/posts/1> .

# File: https://pod.example/bob/posts/1
<https://pod.example/bob/posts/1> a :Post ;
    :content "Hello Solid world!" .

# File: https://pod.example/alice/interests.ttl
<https://pod.example/alice/profile.ttl#me> :interestedIn "Wildlife" .

# File: https://pod.example/forum/wildlife.ttl
<https://pod.example/forum/wildlife.ttl> a :Forum ;
    :hasPost <https://pod.example/ruben/posts/9> ;
    :hasComment <https://pod.example/bob/comment/1> ;
```

```
    :hasTopic "Wildlife" .
<https://pod.example/bob/comment/1> :responseTo <
    https://pod.example/ruben/posts/9> .
```

The generator randomly selects a user and simulates a realistic sequence of queries that reflects real-world user behavior. Given Given this dataset, and two query templates: one to fetch a person's posts and another to retrieve the content of a specific post, the generator can create a query sequence, which can contain multiple logical sessions. For user "alice", we can start the sequence (and first logical session) by executing the first query template (Q1) to discover Bob's posts, as shown in Listing 2.

Listing 2: Initial query (Q1) fetching Bob's posts.

```
SELECT ?post WHERE {
  <https://pod.example/bob/profile.ttl#me> :posts ?
      post .
}
```

The client-side query engine executes this query by traversing to Bob's profile document, returning the result binding: '?post = <https://pod.example/bob/posts/1>'.

To simulate a user clicking on this discovered post, the sequence generator instantiates the next query template dynamically. It extracts the value bound to "?post" from the results of Q1 and injects it as the subject of Q2, as shown in Listing 3.

Listing 3: Dynamic sequence query (Q2) instantiating a variable from Q1.

```
SELECT ?content WHERE {
  <https://pod.example/bob/posts/1> :content ?
      content .
}
```

This example demonstrates that the instantiation of Q2 strictly depends on the execution results of Q1. Because Q2 requires an instantiated post URI from Q1, these queries are directionally dependent; reversing their order leaves the generator with no method to instantiate the initial query parameters. Consequently, structured logical query sequences naturally emerge from these interdependencies, accurately reflecting realistic exploratory user behavior.

Finally, we simulate logical session switching. Drawing from web browsing behavior, we assign each sequence a session-switching probability sampled from a log-normal distribution with a mean of 15% [38, 51]. During sequence generation, a session switch triggers a random choice with equal probability: the engine either starts a new session (using interest-based instantiation) or resumes a previous one (deriving instantiation values from that session's last executed query). To prevent repetitive queries and maintain a uniform distribution of template instantiations, we explicitly prohibit backtracking within our model. Additionally, we maintain a balanced template distribution by dynamically scaling down each template's selection probability proportionally to its overall frequency.

Continuing our running example, a session switch occurs after executing Q2. Instead of continuing sequence generation from the content of Bob's post, the generator terminates the current path and selects a new template to instantiate: a query for posts within a forum.

As shown in Listing 4, the generator selects a wildlife forum as the initial instantiation value. This query requires an entirely separate data path to execute, breaking the dependency chain of the previous queries and representing a clear change in user intent. We are still generating the same *query sequence*, but the session switch introduces a new *logical session* within that sequence.

Listing 4: Independent query (Q3) initiating a session switch to explore a wildlife forum.

```
SELECT ?post WHERE {
  <https://pod.example/forum/wildlife.ttl> :hasPost
      ?post .
}
```

3.1.2. User Interest Modeling

Individuals interact more frequently with users sharing similar interests [40]. To simulate this behavior, we use the underlying LDBC dataset utilized by SolidBench [19], which inherently models realistic user preferences. The LDBC generator, Datagen, produces person profiles containing specific interests, educational backgrounds, and workplaces. It links users based on these demographic and interest overlaps, yielding a non-random network topology with a realistic degree distribution.

We use this interest-driven topology to generate template instantiation values for the first query of each logical session. Specifically, we construct a binary feature vector v_k for each user k:

$$v_k = (x_{k,1}, \ldots, x_{k,n}, y_{k,1}, \ldots, y_{k,m}), \tag{1}$$

where $x_{k,j} = 1$ if user k holds stated interest j, and $y_{k,l} = 1$ if user k completed study l.

For each sequence, we randomly select a base person k and compute the cosine similarity between v_k and the feature vectors of all other users. Let N be the total number of users. To maintain efficiency at large scale factors, we restrict our candidate pool to $C_{k,M}$, defined as the set of the top $M < N$ users ranked by similarity to k. Template instantiation values are then selected probabilistically using a softmax function with temperature T:

$$P(i \mid k) = \frac{e^{s_{k,i}/T}}{\sum_{u \in C_{k,M}} e^{s_{k,u}/T}}, \tag{2}$$

where $s_{k,i}$ is the similarity score between base person k and candidate $i \in C_{k,M}$. For templates requiring non-person values (e.g., posts or activities), selection probabilities are derived from the similarity scores of the content's creators.

Continuing our running example, consider an expanded decentralized environment containing a second forum dedicated to pottery, as shown in Listing 5.

Listing 5: The decentralized dataset augmented with an additional photography forum document.

```
# ... [Previous files from Listing 1 remain
    unchanged] ...

# File: https://pod.example/forum/pottery.ttl
<https://pod.example/forum/pottery.ttl> a :Forum ;
    :hasTopic "Pottery".
```

To determine what instantiation value to use for the forum query for user Alice, we construct binary feature vectors over a small global feature space: (Wildlife, Pottery).

Alice's profile states an explicit interest in 'Wildlife', yielding vector $v_{\text{Alice}} = (1, 0)$. While the forums in the data have topic statements, thus the vectors for the forums are $v_{\text{Wildlife}} = (1, 0)$ and $v_{\text{Pottery}} = (0, 1)$.

This gives us a cosine similarity of $s_{\text{Alice,Wildlife}} = 1$ and $s_{\text{Alice,Pottery}} = 0$. Then using softmax with a temperature of $T = 0.5$, we compute the selection probabilities for the two forums:

$$P(\text{Wildlife} \mid \text{Alice}) = \frac{e^{1/0.5}}{e^{1/0.5} + e^{0/0.5}} \approx 0.88,$$
$$P(\text{Pottery} \mid \text{Alice}) = \frac{e^{0/0.5}}{e^{1/0.5} + e^{0/0.5}} \approx 0.12.$$

Consequently, when initiating or switching a session, the generator is more likely to select the 'Wildlife' forum for Alice, reflecting her interests.

3.1.3. Refinement Patterns

The benchmark simulates query refinement by applying composable transformations to an instantiated query template. Following our literature review, we focus on transformations (and their reciprocal counterparts) that add or modify BGPs, `OPTIONAL` clauses, `UNION` blocks, and query instantiation values.

To ensure these transformations meaningfully alter query execution rather than superficially appending triples, we apply a choke point-based design methodology [4]. The generator sequentially applies refinements to a base SolidSessionBench template. By mapping empirical exploratory usage patterns [5] to concrete planning (P) and traversal (T) choke points, we translate real-world user behaviors into rigorous execution bottlenecks for query engines.

The planning choke points, derived from empirical usage patterns [5] and relevant LDBC choke points [3], are defined as follows:

P.1 Involves adding or removing a `FILTER`, requiring prior knowledge to adjust query plans and estimate selectivity.

P.2 Addresses adding or removing `UNION` or `OPTIONAL` clauses to alter filter push-down capabilities and complicate query planning.

P.3 Covers adding or removing selective triple patterns, forcing the engine to identify changes and re-optimize the plan.

P.4 Focuses on adding or removing non-selective triple patterns.

Beyond planning, these refinements introduce traversal choke points that expand upon those established in SolidBench [56, 55]:

T.1 Expands or contracts the reachable sub-web through predicate additions.

T.2 Shifts the sub-web by substituting the traversal starting point.

T.3 Changes the number of hops required to obtain relevant data, corresponding to SolidBench choke points L.1–L.6 [55].

T.4 Alters the usability of structural indexes, such as type indexes, corresponding to SolidBench choke point L.8 [55].

These choke points directly map to the exploratory query behaviors identified in Section 2.2:

- **Result refinement** corresponds to P.1 and P.3, as it narrows results via `FILTER` constraints and selective triple patterns.
- **Result generalization** corresponds to P.3 and P.4, expanding existing BGPs with selective or non-selective triple patterns.
- **Result extension** maps to P.2, P.3, and P.4 by appending triple patterns to BGPs or introducing `OPTIONAL` and `UNION` blocks that alter filter push-down capabilities.
- **Shifting query intent** corresponds to T.1 and T.2, substituting instantiation values to initiate traversal from different seed documents, thereby shifting or expanding the reachable sub-web.
- **Query refinement** maps to P.2, modifying the structural complexity of the query by adding or removing `OPTIONAL` and `UNION` blocks.

Finally, the generator supports variable instantiation within refinements, allowing users to bind pattern variables to specific values randomly selected from configuration candidates. Users can define custom refinement patterns via JSON files to augment the default configurations.

An example of a slimmed-down default configuration is provided in Listing 6. This entry specifies an `OPTIONAL` refinement pattern that appends a triple pattern to retrieve the messages liked by a person.

The `location` parameter is set to 0, indicating that the triple pattern is added to the first existing `OPTIONAL` block, or to a new block if none is present. The default benchmark suite also includes a reciprocal configuration where `operation` is set to "removal", which instead deletes the triple pattern from the first `OPTIONAL` block.

Table 1
LogT-normal distribution parameters (μ, σ) control the queries per sequence, queries per logical session, and the probability of switching logical sessions. Refinement parameters define the likelihood (`patternProbability`) and bounds (`min/maxRefinementLength`) of generating a refinement sequence for a given query. Instantiation relies on a softmax `temperature` and a `maxLogits` cutoff for similarity-based entity selection.

Category	Parameter	Value
Distributions (μ, σ)	`Sequence Length`	3.0, 0.2
	`Session Length`	1.7, 0.5
	`Transition Probability`	-2.0, 0.25
Refinements	`patternProbability`	0.1
	`min/maxRefinementLength`	2, [3–5]
Instantiation	`temperature`	0.1
	`maxLogits`	5–10

Listing 6: Example configuration for an `OPTIONAL` refinement pattern.

```
{
  "type": "OPTIONAL",
  "id": 0,
  "operation": "addition",
  "description": "Add triple for likes (P.1, T.2)",
  "location": 0,
  "target": [
    {
      "subject": {
        "value": "person",
        "termType": "variable"
      },
      "predicate": {
        "value": ".../vocabulary/likes",
        "termType": "namedNode"
      },
      "object": {
        "value": "likedMessage",
        "termType": "variable"
      }
    }
  ]
}
```

All defaults, along with complete documentation, are available on GitHub.[1]

3.2. Hyperparameters

The sequence generation process introduces several new hyperparameters, which are detailed in Table 1 together with their default values. To guarantee reproducibility despite extensive random sampling, a single seeded pseudo-random number generator (PRNG) is used througout the entire pipeline.

{TODO: Mention hyperparameter evaluation}

[1] https://github.com/SolidBench/SolidBench.js/tree/master/templates/refinements

4. Caching in Link Traversal-based Query Processing

Unlike traditional query engines that evaluate queries against a centralized, pre-built index, LTQP evaluates queries directly over the live Web [55, 29] using the "follow-your-nose" principle.

Starting from a seed URI (often extracted from the query), the engine fetches data via a *link queue* (governed by *reachability criteria* [30]) and deposits the retrieved triples into an *active local store*. This local store is an in-memory dataset over which the query executes, typically utilizing dictionary encoding [54].

Previous work [27] established the fundamental concept of caching within this paradigm, demonstrating that a local repository of recently accessed Web documents mitigates the high network latency inherent to LTQP.

For our experiments, we use the Comunica LTQP engine [54, 56]. As a JavaScript-based engine, it relies on the event loop to asynchronously interleave traversal and local query execution. We validate the SolidSessionBench sequences by integrating three LRU-based caching approaches into this architecture. These approaches cache dereferenced triples in-memory to reduce latency, and we evaluate their impact by comparing three configurations:

Unindexed Cache This baseline approach caches raw arrays of triples alongside source metadata. Upon a cache hit, the engine retrieves and indexes these triples into the active local store.

Indexed Cache This approach pre-indexes triples before caching. Upon a hit, the engine extracts only triples matching the query's patterns. This pre-filtering, combined with dictionary encoding, reduces local store memory and indexing overhead, potentially offsetting the higher initial ingestion costs.

Indexed Cache + Cardinality Estimation This configuration leverages the indexed cache to estimate quad pattern cardinalities across *all* cached documents. This replaces traditional zero-knowledge heuristics [28] with Comunica's cost-based optimizer [54] to generate more accurate query plans.

We evaluate three cache capacities: small (350k quads, ∼10% of the dataset), medium (700k, ∼20%), and large (1.4M, ∼40%).

Our implementations adhere to the RFC-9111 specification [22], meaning the cache correctly interprets standard HTTP headers (e.g., Cache-Control) just like a web browser would. However, because the benchmark dataset is static, cache entries require no revalidation and never become stale. Consequently, observed hit rates are likely higher than in a live environment.

Ultimately, comparing these strategies establishes strong baselines for caching performance in realistic user-centric

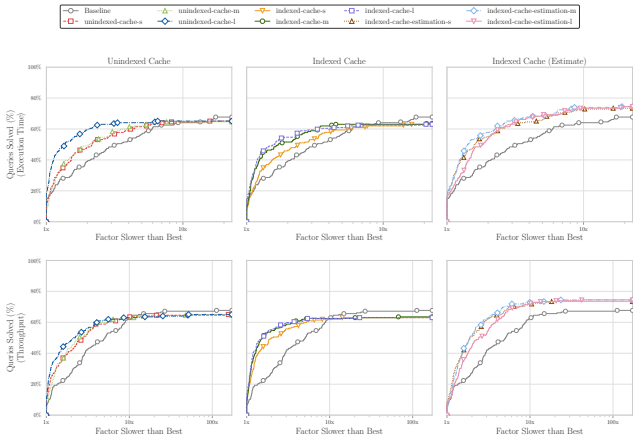


Figure 2: Performance profiles of the tested cache algorithms. The y-axis shows the percentage of queries executing within a factor (x-axis) of the fastest algorithm. Curves closer to the top-left indicate better overall performance. The y-intercept ($x = 1$) shows how often an algorithm is the outright fastest, while the rightward trajectory demonstrates robustness (e.g., $x = 2, y = 80\%$ means 80% of queries execute within twice the fastest time). From the figure, we observe that all caching-based approaches outperform the no-cache baseline.

environments, upon which future research into advanced caching approaches can build.

5. Evaluation

We evaluate our benchmark using a scale 0.1 SolidBench dataset comprising 158,233 RDF files, 1,531 vaults, and 3,556,159 triples. Applying the default hyperparameters from Table 1, we generated 8 sequences totaling 192 interactive-workload queries, encompassing both discover and short templates. Experiments were conducted on an Ubuntu 20.04 machine (Intel Core i5-9400) with 16 GB RAM and a 180-second timeout. To minimize variance and simulate realistic networks, we repeated each sequence 10 times with 70–90 ms injected latency to simulate realistic network conditions. Caches were scoped to individual sequences, ensuring no state was shared between repetitions.

These experiments aim to validate our sequence generation by assessing how our design choices influence query correlation, measured via cache hit rates and evictions, and to establish a caching performance baseline for a Link Traversal-based Query Processing (LTQP) engine. While we report speedups to demonstrate utility, the primary focus is benchmark validation rather than exhaustive algorithm evaluation.

5.1. Global Cache Performance

Figure 2 compares performance profiles [18] for each approach against the no-cache baseline. As expected, all caching methods outperform the baseline.

For the unindexed cache, small (350k triples) and medium (700k triples) sizes perform similarly. The large cache (1,400k triples) significantly improves speed by retaining enough entries across the sequence. However, the baseline resolves more total queries, suggesting cache management overhead causes timeouts for longer queries.

The indexed cache has a higher failure rate, as initial indexing costs slow execution when the eviction rate outweighs the increased efficiency of cache hits. Still, it accelerates successful queries more than the unindexed cache. It also reaches critical capacity sooner, with medium and large sizes performing similarly.

Overall, the indexed cache with cardinality estimation performs best, increasing speed and solving more queries. Notably, larger cache sizes degrade performance. A possible mechanism for this is that the estimator evaluates all cache entries, including irrelevant unreachable data from past queries. This accumulated noise likely can degrade estimates, suggesting the estimator favors smaller, fresher caches.

5.2. Logical Sessions

To evaluate the impact of logical sessions on caching performance, we analyze the hit rate deviation (Δ) from the overall average when queries initiate a new session, switch to an existing one, or remain in the current session. Table 2 shows that new sessions severely degrade hit rates, existing session switches cause moderate degradation, and intra-session queries improve performance. These effects are strongest in small and medium caches.

Consequently, optimal client-side cache sizing depends heavily on user session-switching behavior. These findings highlight the need for eviction policies that better retain cross-session entries and validate our simulation design; ignoring session dynamics would obscure critical real-world performance shifts.

Future work should investigate whether query sequences should share a frequently accessed core of cache entries. Protecting these entries from eviction could mitigate the performance penalties of session switching. Engines could also attempt to identify sessions and keep seperate caches for each session.

5.3. Refinement Patterns

To evaluate iterative query refinement simulation, we analyze cache behavior across refinement depths. Figure 3 shows that hit ratios increase progressively as queries undergo sequential refinement. Across all algorithms, hit rates increase at depths 2 and 3, confirming that refinement heavily reuses prior data.

At higher refinement depths, performance depends heavily on cache capacity. Large caches sustain high hit rates through depth 4, whereas small caches experience significant thrashing, and degraded hit ratios.

Surprisingly, the indexed cache without estimation maintains high hit rates at small capacities. Different caching mechanisms alter how the Node.js event loop schedules traversal and join executions, changing traversal order and rate. We suspect this variance increases temporal locality, though further analysis is needed to confirm the exact

Table 2
Impact of logical session status on algorithm performance. Values represent absolute and percentage cache hit rate deviations from the global template average across new, existing, and within-session queries. Negative values mean caches perform worse for these types of queries.

Cache Algorithm	New Session		Existing Session		Within Session	
	Abs.	%	Abs.	%	Abs.	%
unindexed-l	-0.066	-12.278	-0.014	-2.474	0.011	1.931
unindexed-m	-0.056	-15.649	-0.031	-7.900	0.024	5.486
unindexed-s	-0.071	-24.275	-0.021	-6.551	0.017	4.582
indexed-estimate-l	-0.053	-10.573	-0.015	-2.796	0.012	2.127
indexed-estimate-m	0.028	7.174	-0.000	-0.064	-0.000	-0.035
indexed-estimate-s	-0.044	-14.305	-0.013	-3.921	0.010	2.924
indexed-l	-0.026	-5.011	-0.003	-0.571	0.003	0.467
indexed-m	-0.087	-22.782	-0.021	-5.098	0.017	3.682
indexed-s	-0.033	-11.036	-0.015	-4.470	0.012	3.153

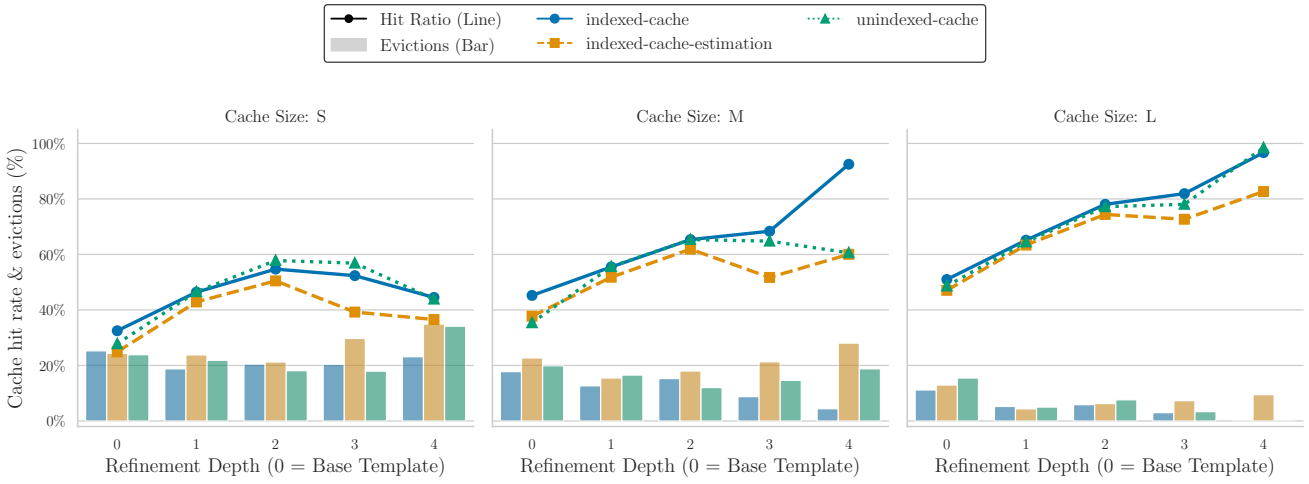


Figure 3: Cache hit ratios and percentage of cache capacity evicted across refinement depths for small, medium, and large cache configurations.

mechanism and explain why the indexed cache outperforms the unindexed baseline.

The observed cache hit rate and eviction differences directly translate to measurable execution speedups. Figure 2 illustrates that the geometric mean speedup generally scales positively with refinement depth, with a more pronounced effect for the throughput. The indexed-cache algorithm proves particularly effective for prolonged exploratory sequences, achieving substantial throughput gains at depth 4 compared to the unindexed alternatives.

5.4. Comparison To Other Evaluation Methods

Microbenchmarks - Theoretical Hit Rate Analysis Cache literature extensively uses microbenchmarks of various forms to analyze caching performance [41, 27]. To compare the conclusions of such a benchmark to the conclusions of the benchmark introduced in this paper we investigate theoretical caching speedup without management costs, by executing each query template twice with simulated cache miss probabilities. We excluded the estimation-based approach, as using the full cache for estimation would skew the results for miss rates above zero.

Figure 5 shows representative performance profiles for the workload. Most templates (8 out of 12) show reliable improvement that scales with the hit rate for both algorithms. However, certain long-running queries (e.g., interactive-discover-6, 7, and short-3) show no speedup, and some even experience slowdowns. These templates traverse many solid pods but do not require all traversed data. We hypothesize that rapidly serving documents from an in-memory cache overwhelms the system with data, slowing result production. Conversely, HTTP network latency gives the JavaScript event loop time to process the join execution pipeline, yielding initial results faster since only the first few requests are needed.

Both caching approaches perform similarly overall, but the indexed cache significantly reduces time-to-first-result across most templates. As expected, indexed caching is highly advantageous without evictions. However, the initial indexing cost creates a trade-off: it amplifies both speedups

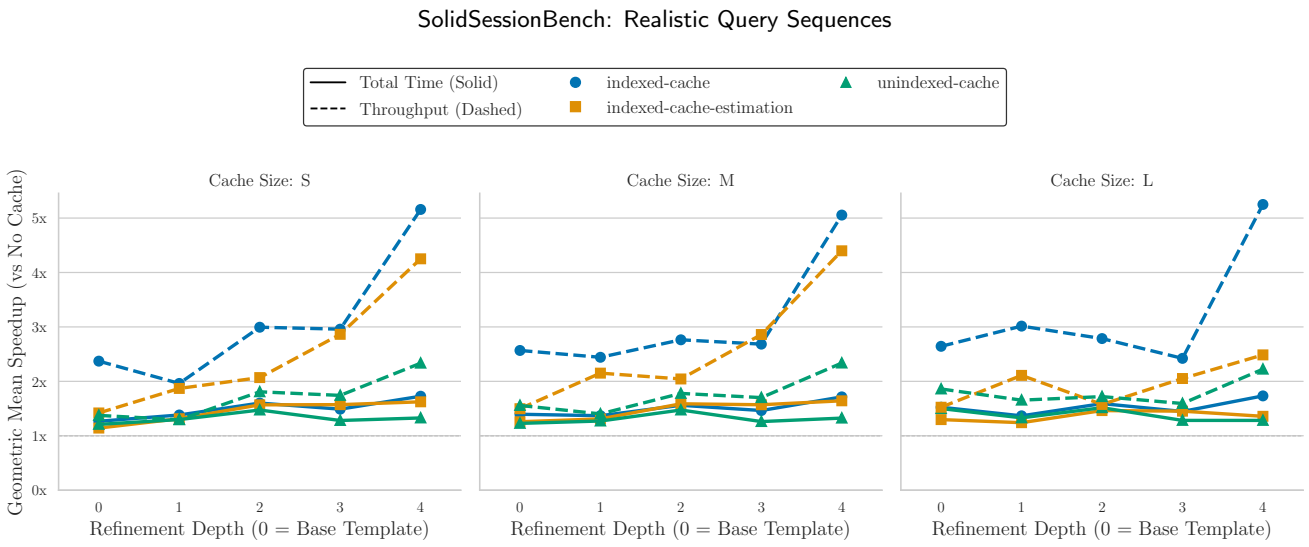


Figure 4: Geometric mean of observed speedups and increases respectively for total execution time and query throughput relative to the no caching baseline across refinement depths.

Figure 5: Performance of unindexed and indexed caches across representative query templates. The y-axis shows the mean speedup relative to a baseline with a 0% hit rate. Shaded regions denote the 90th percentile

and slowdowns relative to an unindexed cache. While this micro-benchmark approach aligns with prior literature [41], our mixed findings show that micro-benchmarks alone fail to thoroughly evaluate client-side caching optimizations. This limitation demonstrates the clear need for the comprehensive benchmark we introduce in this work.

Random Query Sequences Another common approach is to use random query sequences from an existing benchmark [42, 11, 27, 41, 23]. Figure 6 shows execution times for sequences created by randomly grouping default SolidBench.js queries. For these random sequences, only large caches perform well, while smaller ones degrade below the baseline. These results misleadingly suggest that large caches are vital, whereas our benchmark demonstrates significant performance benefits even with smaller sizes. As expected, the estimation cache performs well in both scenarios because cardinality estimation does not require cache hits.

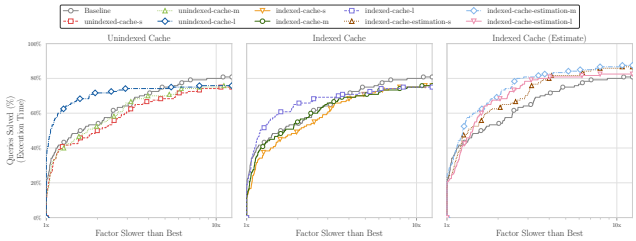


Figure 6: Performance profile of the tested cache algorithms on random query sequences. The y-axis shows the percentage of queries that execute within a given performance factor (x-axis) of the fastest algorithm for that query.

Scope and Limitations We evaluate SolidSessionBench against microbenchmarks and random sequences, but explicitly exclude hand-crafted scenarios and simple distributions. Hand-crafted methods lack scalability, while simple distributions fail to capture complex user habits. By automating logical sessions and user interests, our benchmark functions as a scalable superset of both approaches, making direct comparison redundant. By contrast, the compared methods do not incorporate these sequence attributes, and our results show this omission risks drawing inaccurate conclusions.

6. Conclusion

We introduced SolidSessionBench, an evidence-based benchmark for generating realistic query sequences in decentralized environments. By modeling logical sessions, user interests, and iterative refinement, it captures client-side performance characteristics that traditional evaluation methods fail to find. While using Solid as the primary use case, the underlying concepts apply to approaches such as Linked Web Storage (LWS), and we will provide an LWS-specific representation once the W3C Working Group finalizes the specifications.

To ensure longevity, SolidSessionBench has been integrated into the core SolidBench ecosystem, inheriting its established track record of community support and maintenance. We formalize this in our sustainability plan `https://github.com/SolidBench/SolidBench.js/wiki/Sustainability-Plan`. Due to the seamless integration into SolidBench, its status as the standard community resource for evaluating decentralized query execution [20, 59, 26, 57] serves as a proxy for the future adoption of this new benchmark when trying to evaluate any client-specific optimization approaches.

The framework is modular, MIT-licensed, and fully documented, with links to all code, data, and experiment setups available at this paper-specific hub: `https://github.com/RubenEschauzier/simulated-query-sequence-paper`. This extension-friendly approach to development allows researchers to easily modify or extend specific components to meet future requirements without disrupting the broader evaluation pipeline.

7. Declaration of Generative AI Use

In this work, we used Gemini to:

- Generate self-contained code snippets, which we validated through manual and automated testing.
- Generate code to produce figures.
- Identify and resolve bugs.
- Rewrite and condense paragraphs to improve clarity.

The authors reviewed all AI-generated content and take full responsibility for the final manuscript and codebase.

CRediT authorship contribution statement

Ruben Eschauzier: TODO:. **Ruben Taelman:** TODO:.

References

[1] Aluç, G., Hartig, O., Özsu, M.T., Daudjee, K., 2014. Diversified stress testing of rdf data management systems, in: International Semantic Web Conference, Springer. pp. 197–212.

[2] Amblard, F., Bouadjio-Boulic, A., Gutiérrez, C.S., Gaudou, B., 2015. Which models are used in social simulation to generate social networks? a review of 17 years of publications in jasss, in: 2015 Winter Simulation Conference (WSC), IEEE. pp. 4021–4032.

[3] Angles, R., Antal, J.B., Averbuch, A., Birler, A., Boncz, P., Búr, M., Erling, O., Gubichev, A., Haprian, V., Kaufmann, M., et al., 2020. The ldbc social network benchmark.

[4] Angles, R., Boncz, P., Larriba-Pey, J., Fundulaki, I., Neumann, T., Erling, O., Neubauer, P., Martinez-Bazan, N., Kotsev, V., Toma, I., 2014. The linked data benchmark council: a graph and rdf industry benchmarking effort, ACM New York, NY, USA. pp. 27–31.

[5] Arenas, M., Franconi, E., Hammerer, J., Hartig, O., Hose, K., Koesten, L., Konstantinidis, G., Libkin, L., Martens, W., Sasaki, Y., et al., 2025. Exploring exploratory querying. Proceedings of the VLDB Endowment 18, 5731–5739.

[6] Arnold, B.C., 2014. Pareto distribution. Wiley StatsRef: Statistics Reference Online , 1–10.

[7] Backstrom, L., Huttenlocher, D., Kleinberg, J., Lan, X., 2006. Group formation in large social networks: membership, growth, and evolution, in: Proceedings of the 12th ACM SIGKDD international conference on Knowledge discovery and data mining, pp. 44–54.

[8] Bast, H., Buchhold, B., 2017. Qlever: A query engine for efficient sparql+ text search, in: Proceedings of the 2017 ACM on Conference on Information and Knowledge Management, pp. 647–656.

[9] Berendt, B., Hollink, L., Luczak-Rösch, M., Möller, K., Vallet, D., 2013. Usewod2013 3rd international workshop on usage analysis and the web of data. 10th ESWC-Semantics and Big Data, Montpellier, France .

[10] Berners-Lee, T., 2017. Charlie: An AI that works for you. W3C Design Issues. URL: `https://www.w3.org/DesignIssues/Charlie.html`. updated 2023.

[11] Bizer, C., Schultz, A., 2011. The berlin sparql benchmark, in: Semantic Services, Interoperability and Web Applications: Emerging Concepts. IGI Global Scientific Publishing, pp. 81–103.

[12] Bonifati, A., Martens, W., Timm, T., 2018. DARQL: Deep Analysis of SPARQL Queries, in: Companion Proceedings of the The Web Conference 2018, International World Wide Web Conferences Steering Committee, Republic and Canton of Geneva, CHE. pp. 187–190. URL: `https://dl.acm.org/doi/10.1145/3184558.3186975`, doi:`10.1145/3184558.3186975`.

[13] Bonifati, A., Martens, W., Timm, T., 2020. An analytical study of large sparql query logs. The VLDB Journal 29, 655–679.

[14] Bronson, N., Amsden, Z., Cabrera, G., Chakka, P., Dimov, P., Ding, H., Ferris, J., Giardullo, A., Kulkarni, S., Li, H., et al., 2013. {TAO}:{Facebook's} distributed data store for the social graph, in: 2013 USENIX annual technical conference (USENIX ATC 13), pp. 49–60.

[15] Chatzopoulos, S., Vergoulis, T., Skoutas, D., Dalamagas, T., Tryfonopoulos, C., Karras, P., 2022. Atrapos: Evaluating metapath query workloads in real time. CoRR .

[16] Clark, A., 2007. Understanding community: A review of networks, ties and contacts .

[17] Dang, M.H., Aimonier-Davat, J., Molli, P., Hartig, O., Skaf-Molli, H., Le Crom, Y., 2023. Fedshop: A benchmark for testing the scalability of sparql federation engines, in: International Semantic Web Conference, Springer. pp. 285–301.

[18] Dolan, E.D., Moré, J.J., 2002. Benchmarking optimization software with performance profiles. Mathematical programming 91, 201–213.

[19] Erling, O., Averbuch, A., Larriba-Pey, J., Chafi, H., Gubichev, A., Prat, A., Pham, M.D., Boncz, P., 2015. The ldbc social network benchmark: Interactive workload, in: Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data, pp. 619–630.

[20] Eschauzier, R., Taelman, R., Verborgh, R., 2025. Revisiting link prioritization for efficient traversal in structured decentralized environments, in: International Semantic Web Conference, Springer. pp. 575–593.

[21] Esteves, B., Pandit, H.J., Rodríguez-Doncel, V., 2021. Odrl profile for expressing consent through granular access control policies in solid, in: 2021 IEEE European Symposium on Security and Privacy Workshops (EuroS&PW), IEEE. pp. 298–306.

[22] Fielding, R., Nottingham, M., Reschke, J., 2022. Rfc 9111: Http caching.

[23] Folz, P., Skaf-Molli, H., Molli, P., 2016. Cyclades: a decentralized cache for triple pattern fragments, in: European semantic web conference, Springer. pp. 455–469.

[24] Hamill, L., Gilbert, G., 2009. Social circles: A simple structure for agent-based social network models. Journal of Artificial Societies and Social Simulation 12.

[25] Hanski, J., Taelman, R., Verborgh, R., 2024. Observations on bloom filters for traversal-based query execution over solid pods, in: European Semantic Web Conference, Springer. pp. 228–233.

[26] Hanski, J., Van Braeckel, S., Taelman, R., Verborgh, R., 2025. Link traversal over decentralised environments using restart-based query planning, in: International Conference on Web Engineering.

[27] Hartig, O., 2011a. How caching improves efficiency and result completeness for querying linked data., in: LDOW.

[28] Hartig, O., 2011b. Zero-knowledge query planning for an iterator implementation of link traversal based query execution, in: Extended Semantic Web Conference, Springer. pp. 154–169.

[29] Hartig, O., 2012. Sparql for a web of linked data: Semantics and computability, in: Extended Semantic Web Conference, Springer. pp. 8–23.

[30] Hartig, O., Bizer, C., Freytag, J.C., 2009. Executing sparql queries over the web of linked data, in: International semantic web conference, Springer. pp. 293–309.

[31] Hartig, O., Özsu, M.T., 2016. Walking without a map: Ranking-based traversal for querying linked data, in: International Semantic Web Conference, Springer. pp. 305–324.

[32] Heling, L., Acosta, M., 2022. Federated sparql query processing over heterogeneous linked data fragments, in: Proceedings of the ACM Web Conference 2022, pp. 1047–1057.

[33] Kalaitzakis, A., Papadakis, H., Fragopoulou, P., 2012. Evolution of user activity and community formation in an online social network, in: Proceedings of the 2012 IEEE/ACM International Conference on Advances in Social Networks Analysis and Mining, IEEE. pp. 1315–1320.

[34] Karki, N., Pandey, M., Tiwari, S., Mihindukulasooriya, N., Groppe, S., Dobriy, D., 2026. Agentic ai, context engineering and knowledge graphs: Current approaches, challenges and opportunities., in: EDBT/ICDT Workshops.

[35] Lorey, J., Naumann, F., 2013. Caching and prefetching strategies for sparql queries, in: Extended Semantic Web Conference, Springer. pp. 46–65.

[36] Malyshev, S., Krötzsch, M., González, L., Gonsior, J., Bielefeldt, A., 2018. Getting the most out of wikidata: Semantic technology usage in wikipedia's knowledge graph, in: The Semantic Web–ISWC 2018: 17th International Semantic Web Conference, Monterey, CA, USA, October 8–12, 2018, Proceedings, Part II 17, Springer. pp. 376–394.

[37] Martin, M., Unbehauen, J., Auer, S., 2010. Improving the performance of semantic web applications with sparql query caching, in: Extended Semantic Web Conference, Springer. pp. 304–318.

[38] Meiss, M., Duncan, J., Gonçalves, B., Ramasco, J.J., Menczer, F., 2009. What's in a session: tracking individual behavior on the web, in: Proceedings of the 20th ACM conference on Hypertext and hypermedia, ACM, Torino Italy. pp. 173–182. URL: https://dl.acm.org/doi/10.1145/1557914.1557946, doi:10.1145/1557914.1557946.

[39] Mislove, A., Marcon, M., Gummadi, K.P., Druschel, P., Bhattacharjee, B., 2007. Measurement and analysis of online social networks, in: Proceedings of the 7th ACM SIGCOMM conference on Internet measurement, pp. 29–42.

[40] Mitzlaff, F., Atzmueller, M., Benz, D., Hotho, A., Stumme, G., 2013. User-relatedness and community structure in social interaction networks. arXiv preprint arXiv:1309.3888 .

[41] Nicholson, H., Chrysogelos, P., Ailamaki, A., 2024. Hpcache: memory-efficient olap through proportional caching revisited. The VLDB Journal 33, 1775–1791.

[42] O'Neil, P., O'Neil, E., Chen, X., Revilak, S., 2009. The star schema benchmark and augmented fact table indexing, in: Technology Conference on Performance Evaluation and Benchmarking, Springer. pp. 237–252.

[43] Padia, A., Finin, T., Joshi, A., et al., 2015. Attribute-based fine grained access control for triple stores, in: 3rd Society, Privacy and the Semantic Web-Policy and Technology workshop, 14th International Semantic Web Conference.

[44] Papailiou, N., Tsoumakos, D., Karras, P., Koziris, N., 2015. Graph-aware, workload-adaptive sparql query caching, in: Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data, pp. 1777–1792.

[45] Picalausa, F., Vansummeren, S., 2011. What are real SPARQL queries like?, in: Proceedings of the International Workshop on Semantic Web Information Management, Association for Computing Machinery, New York, NY, USA. pp. 1–6. URL: https://dl.acm.org/doi/10.1145/1999299.1999306, doi:10.1145/1999299.1999306.

[46] Raman, A., Joglekar, S., Cristofaro, E.D., Sastry, N., Tyson, G., 2019. Challenges in the decentralised web: The mastodon case, in: Proceedings of the internet measurement conference, pp. 217–229.

[47] Sadagopan, N., Li, J., 2008. Characterizing typical and atypical user sessions in clickstreams, in: Proceedings of the 17th international conference on World Wide Web, pp. 885–894.

[48] Saleem, M., Ali, M.I., Hogan, A., Mehmood, Q., Ngomo, A.C.N., 2015. Lsq: the linked sparql queries dataset, in: International semantic web conference, Springer. pp. 261–269.

[49] Sambra, A.V., Mansour, E., Hawke, S., Zereba, M., Greco, N., Ghanem, A., Zagidulin, D., Aboulnaga, A., Berners-Lee, T., 2016. Solid: a platform for decentralized social applications based on linked data. MIT CSAIL & Qatar Computing Research Institute, Tech. Rep. 2016.

[50] Schmidt, M., Görlitz, O., Haase, P., Ladwig, G., Schwarte, A., Tran, T., 2011. Fedbench: A benchmark suite for federated semantic data query processing, in: The Semantic Web–ISWC 2011: 10th International Semantic Web Conference, Bonn, Germany, October 23-27, 2011, Proceedings, Part I 10, Springer. pp. 585–600.

[51] Schneider, F., Feldmann, A., Krishnamurthy, B., Willinger, W., 2009. Understanding online social network usage from a network perspective, in: Proceedings of the 9th ACM SIGCOMM Conference on Internet Measurement, pp. 35–48.

[52] Schwarte, A., Haase, P., Hose, K., Schenkel, R., Schmidt, M., 2011. Fedx: Optimization techniques for federated query processing on linked data, in: The Semantic Web–ISWC 2011: 10th International Semantic Web Conference, Bonn, Germany, October 23-27, 2011, Proceedings, Part I 10, Springer. pp. 601–616.

[53] Stadler, C., Saleem, M., Mehmood, Q., Buil-Aranda, C., Dumontier, M., Hogan, A., Ngonga Ngomo, A.C., 2024. Lsq 2.0: A linked dataset of sparql query logs. Semantic Web 15, 167–189.

[54] Taelman, R., Van Herwegen, J., Vander Sande, M., Verborgh, R., 2018. Comunica: a modular sparql query engine for the web, in: International Semantic Web Conference, Springer. pp. 239–255.

[55] Taelman, R., Verborgh, R., 2023a. Evaluation of link traversal query execution over decentralized environments with structural assumptions.

[56] Taelman, R., Verborgh, R., 2023b. Link traversal query processing over decentralized environments with structural assumptions, in: International Semantic Web Conference, Springer. pp. 3–22.

[57] Tam, B.E., Taelman, R., Colpaert, P., Verborgh, R., 2024. Opportunities for shape-based optimization of link traversal queries. arXiv preprint arXiv:2407.00998 .

[58] Umbrich, J., Hose, K., Karnstedt, M., Harth, A., Polleres, A., 2011. Comparing data summaries for processing live queries over linked data. World Wide Web 14, 495–544.

[59] Vandenbrande, M., Taelman, R., Bonte, P., Ongenae, F., 2025. Incremunica: Web-based incremental view maintenance for sparql, in: European Semantic Web Conference, Springer. pp. 297–315.

[60] Vögele, C., van Hoorn, A., Schulz, E., Hasselbring, W., Krcmar, H., 2018. Wessbas: extraction of probabilistic workload specifications for load testing and performance prediction—a model-driven approach for session-based application systems. Software & Systems Modeling 17, 443–477.

[61] Walter, S., Bast, H., 2025. Grasp: Generic reasoning and sparql generation across knowledge graphs, in: International Semantic Web Conference, Springer. pp. 271–289.

[62] Williams, G.T., Weaver, J., 2011. Enabling fine-grained http caching of sparql query results, in: International Semantic Web Conference, Springer. pp. 762–777.

[63] Zhang, W.E., Sheng, Q.Z., Qin, Y., Yao, L., Shemshadi, A., Taylor, K., 2016. Secf: improving sparql querying performance with proactive fetching and caching, in: Proceedings of the 31st Annual ACM Symposium on Applied Computing, pp. 362–367.

[64] Zhang, X., Wang, M., Zhao, B., Liu, R., Zhang, J., Yang, H., 2020. Characterizing robotic and organic query in sparql search sessions, in: Web and Big Data: 4th International Joint Conference, APWeb-WAIM 2020, Tianjin, China, September 18-20, 2020, Proceedings, Part I 4, Springer. pp. 270–285.